Chapter 15
VLSI CAD Tools

In this chapter we describe the various steps in the design flow of an integrated circuit, starting from the algorithmic description of the functionality, to its representation and model in a High Level *hardware description language*(HDL). For each step we present open-source VLSI CAD tools. We limit our attention to standard cell based design methodology, but we should point out that in addition to this style there are these other techniques which are also used:

1. Full-custom: the complete chip is directly drawn as a layout. This is useful for analog and radio-frequency (RF) block design. It is also used in custom bus design where the performance of the block is critical to the chip,
2. Data-path design: used in arithmetic blocks where similar operations happen on a wide-bus.
3. Memory array: memory arrays are often designed as a single *bit*-cell, which is then regularly placed by automated tools,
4. PLA design: control logic can be represented as *sum-of-products*(SOP), and PLA (programmable logic arrays) can be automatically drawn from SOP functions. The array is not run-time programmable, only that the choice of which products to sum is made by switches which are programmed when the PLA is drawn.
5. FPGA design:field-programmable gate arrays use arrays of logic elements (LEs) connected by a fabric of programmable routing. Some of the problems, such as floorplanning, and placement can be put in FPGA context as well.

VLSI design has been the study of automation and computer science, and many tools have been written to solve one or more problems associated with the VLSI design flow. Thus a number of standard (and not so standard) *file-formats* have emerged to communicate data from one step of the flow to another.

We first describe the syntax of the Berkeley Logic Interchange Format, and present several tools to analyze BLIF data, as BLIF is a standardized logic interchange format for synthesis tools. Post synthesis the logic design has been converted into a *gate-level* netlist we discuss the problem of timing estimation, buffer-insertion and logic checking. This part of the flow is traditionally known as *front-end* of the VLSI flow, and the timing correct gate-level netlist was the accepted *handoff* to

S. Koranne, *Handbook of Open Source Tools*,
DOI 10.1007/978-1-4419-7719-9_15, © Springer Science+Business Media, LLC 2011

the physical design flow to follow. The major design flow stages are (i) front-end design and HDL capture, (ii) synthesis to gate level netlist, (iii) floorplanning, (iv) placement, (v) global routing, (vi) detailed routing, and (vii) mask level processing.

15.1 Algorithmic Design and HDL Capture

We state the title of this section as "algorithmic design", which refers to the task of solving a given problem by using an algorithm. Consider the problem of designing an elevator control module which services four elevators in a building with ten floors. The requirement analysis of this problem in detail may produces a list as follows:

1. Interact with sensors, lighting and control switches inside the elevator, in the floor and the main control room of the building,
2. Minimize the number of physical connections to the unit,
3. Minimize wait times,
4. Conserve power by maximizing idle item,

A system architect will perform a Pareto analysis of various solutions to this problem. He may consider using an off-the-shelf micro-controller (e.g, 8051,or i486), program an FPGA, design a custom ASIC (chip). The architect will then proceed to design an algorithm to solve the presented problem (within the constraints placed by the system requirements). This algorithm is then *captured* in an HDL for simulation and synthesis. Simulation is yet another area of VLSI CAD which is very important, but we have side-stepped, as writing a correct simulator for a modern HDL is non-trivial. By simulating the HDL of the elevator-controller module, its behavior under various conditions can be tested, and its properties measured. CAD tools to calculate expected power dissipation are also available, and expected wait-times can be checked with random initial conditions. The controller's response to emergency inputs under various states and its guarantee of a maximum service time per request can be checked formally using a technique known as *model checking*, which is derived from finite-state-machine state exploration.

15.2 HDL Capture

Common HDLs are Verilog and VHDL. Both offer similar features in terms of specifying the structure of the logic module, and its behavior using (i) combinational, and (ii) sequential processing elements. Combinational elements are logic circuits which have no state, and their outputs depend on their current inputs *only*. There may still be a logic-delay between the time an input changes, and the time this change is reflected in a change to the output. Combinational logic circuits are manipulated using Boolean algebra, and their properties have been extensively studied and un-

derstood. Sequential processing elements have *state*, and thus, their current output depends not only on their input, but also the current state of the element. This make sequential analysis more complex, but it also allows for a significant reduction in the size of the Boolean circuit needed to implement a given algorithm. For example, 16-bit multiplication would need thousands of logic elements to produce a valid output given only combinational element, but a sequential process combined with combinational logic can perform the same computation (albeit in multiple clock cycles) in only few hundred elements. Sequential elements are often modeled as *state-machines*, and are represented in Verilog/VHDL as encoded state-registers. Consider the following Verilog HDL fragment:

```
module ElevatorControl( clk, rst, swButton,
                        elvButton, emgButton,
                        sensor, opControl);
input    clk,rst;
input [31:0] swButton; //switches in floor, control room
input [63:0] elvButton;// switches inside  4 elevators
...
output [31:0] opControl;// motor control switch lights
endmodule;
```

Although automated tools to convert a given algorithmic description (which may have been done using another language) to HDLs exist, most of the time, the HDL is written by hand in conjunction with the system architect. Key system parameters, such as, interface registers, IO register locations, interface pin-diagrams, need to be created and frozen. Many of these physical parameters have no equivalence in the algorithmic description (what is the weight of an algorithm ?, the controller once manufactured will have weight and dimensions which need to be specified so that the rest of the system can be built around the controller).

As we mentioned earlier, HDL descriptions are simulated and analyzed for their properties. Once the designers are satisfied that the design meets the functional requirements, the design is *synthesized*. Synthesis is the process of taking HDL descriptions, converting them to Boolean logic expressions (using combinational and sequential elements), and finally writing out an equivalent Boolean description of the circuit with each logic elements *mapped* to a given primitive logic element. The set of primitives is also given to the synthesis tool, and this element set is called a *library*. Each library element has been designed to have a physical representation on the chip, and once the circuit is mapped onto the library elements, it can be manufactured purely as a connected graph of these library elements. Each library element provides the synthesis tool the *single* (for single-output standard cells) Boolean function it can implement. It is the task of the synthesis tool to *map* the given Boolean circuit using these primitives. Obviously, an universal primitive like two-input NAND can be used to map the complete combinational portion of the circuit, but it has been known that providing higher levels of Boolean primitives (for example, primitives which compute AND-OR-INV, or AOI, combinations of their input, not only reduce the number of total primitives used, but also reduce the maximum depth of the circuit). The maximum depth of the combinational circuit

determines the clock cycle of the chip, as each clock cycle must leave enough time for the output of every combinational block in the chip to be computed.

Synthesis tools have a rich history, and UC Berkeley provided many of the fundamental results which are now part of every synthesis CAD tool. Thus it is only fitting that we present BLIF (Berkeley Logic Interchange Format) syntax before continuing on to the next steps of the VLSI flow.

15.3 BLIF Format in a nutshell

The BLIF format describes a circuit in terms of models. A model is an arbitrary combinational or sequential network of logic functions. A circuit is built up of modules connected together to form a connected, directed graph. Each cyclic edge in this network must go through a *latch* element. Each net has a single driver, which must be named without ambiguity. A model is declared in BLIF as

```
.model <name>
.inputs <input-list>
.outputs <output-list>
<command>
...
<command>
.end
```

The *command* is one of:

1. logic gate
2. generic latch
3. library-gate: library format is described below,
4. model reference:
5. subfile reference:
6. fsm-description:
7. clock constraint
8. delay constraint

A # begins a comment line which continues to the end of the line. A *logic gate* associates a logic function with a signal in the model, which can be used as an input to another logic function. A logic gate is declared as follows:

```
.names <in-1> <in-2> ... <in-n> <output>
<single-output-cover>
```

An example of a logic-gate is shown below:

```
.names x y z
1- 1
01 1
```

The single-output cover may include – to represent a don't care. Library gates are specified using *.gate* construct, where the syntax is as follows:

```
.gate <name> <formal-argument>
```

The argument list of the gate is looked up from the library file, and is matched by name.

Consider a very small model represented as a BLIF file:

```
.model small
.inputs ck  i0 i1 i2 i3
.outputs  o0 o1
.names  i0 i1 i2 i3 o0
0001 1
0000 1
0010 1
.names  i0 i1 i2 i3 o1
0001 1
0000 1
0010 1
.end
```

This model has 4 inputs (i0,i1,i2 and i3), and 2 outputs (o0 and o1). The truth table (or single-output cover) for o0 and o1 is given in the BLIF file.

Next we write a library description file containing the four gates of INV, BUF, 2-AND and 2-OR. This is shown below:

```
GATE inv_x1 1  q=!i;
     PIN    i  INV 1 999 1.00 0.00 1.00 0.00
GATE buf_x2 1  q=i;
     PIN    i  INV 1 999 1.00 0.00 1.00 0.00
GATE a2_x2  2  q=(i0*i1);
     PIN   i0  INV 1 999 1.00 0.00 1.00 0.00
     PIN   i1  INV 1 999 1.00 0.00 1.00 0.00
GATE o2_x2  2  q=(i0+i1);
     PIN   i0  INV 1 999 1.00 0.00 1.00 0.00
     PIN   i1  INV 1 999 1.00 0.00 1.00 0.00
```

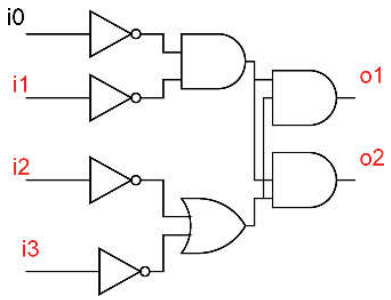


Fig. 15.1 Gate level netlist, schematic is shown after mapping onto library.

Using ABC Synthesis tool we map the BLIF file using the library to produce a gate-level mapped netlist as shown below. The ABC commands necessary to do the mapping were: (i) read_blif (ii) read_library, (iii) map, and (iv) write_blif. We get the following BLIF file.

```
.model small
.inputs ck i0 i1 i2 i3
.outputs o0 o1
.gate inv_x1 i=i2 q=n7
.gate inv_x1 i=i3 q=n8
.gate o2_x2  i0=n8 i1=n7 q=n9
.gate inv_x1 i=i0 q=n10
.gate inv_x1 i=i1 q=n11
.gate a2_x2  i0=n11 i1=n10 q=n12
.gate a2_x2  i0=n12 i1=n9 q=o0
.gate a2_x2  i0=n12 i1=n9 q=o1
.end
```

Once the following gate is introduced in the library, the number of elements and the critical path get reduced.

```
GATE  noa22_x1  3   nq=((!i0+!i1)*!i2);
      PIN     i0 INV 1 999 1.00 0.00 1.00 0.00
      PIN     i1 INV 1 999 1.00 0.00 1.00 0.00
      PIN     i2 INV 1 999 1.00 0.00 1.00 0.00
```

We again map the original input BLIF using the augmented library to get the following BLIF file.

```
.model small
.inputs ck i0 i1 i2 i3
.outputs o0 o1
.gate o2_x2    i0=i1 i1=i0 q=n7
.gate noa22_x1 i0=i3 i1=i2 i2=n7 nq=o0
.gate noa22_x1 i0=i3 i1=i2 i2=n7 nq=o1
.end
```

Thus, it is obvious, how large an impact, choosing a good set of primitive elements in the library can have on the size of the chip and its critical path.

This gate level netlist is the traditional *hand-off* from front-end design tools to the *back-end* processing, also known as the *physical design flow*. In the physical design flow, the netlist is first divided into block regions at the chip level using an automated tool called the *floorplanner*. The input to this tool is the bounding box of each module of the netlist, and it calculates relative positions of the modules to minimize the overall size of this chip.

Once the floorplan is complete, we perform *cell-placement*. Standard cell placement has been an extensively researched topic, with recent publications (as late as Jan. 2009) reporting break-throughs in technology. We present simple, but effective

and time-tested placement solutions based on simulated annealing, and spectral partitioning. Both methods are fully described earlier in the context of graph partitioning (Breuer had shown in the 1970s an interesting relationship between placement and partitioning).

Placement is followed by *global routing*, a process, in which we assign nets to *channels*, which are defined as pathways on the grid of the chip with exclusive reservation of a limited number of nets (to avoid congestion when we perform final detailed routing). An obvious optimization criterion is the minimization of wirelength used to connect any net, but secondary objectives, such as minimizing via-counts (a *via* occurs whenever a wire changes direction on a net), minimizing coupling-length between long and *crosstalk sensitive* wires, and minimizing skew. In our formulation of global routing we have used wirelength as the primary objective, with via-minimization and coupling analysis and minimization as the secondary objectives. Our formulation of global routing is based on *flows* in the grid-graph.

Global routing is followed by detailed routing. Detailed routing has developed into its own industry as it needs to cater to foundry specific design rules, timing calculations, and yield analysis. Thus, we shall not present any detailed router in this text (it is fit to be the topic of a book by itself).

A completely routed design database is processed at the *mask shop* where image processing of the data is done to verify printability on the wafer. This is done using a combination of image processing tasks such as optical proximity correction, resolution enhancement and sub-resolution assist features. Like detailed routing, mask optimization is also an industry within its own right, and we defer on that topic as well.

15.4 Schematic capture

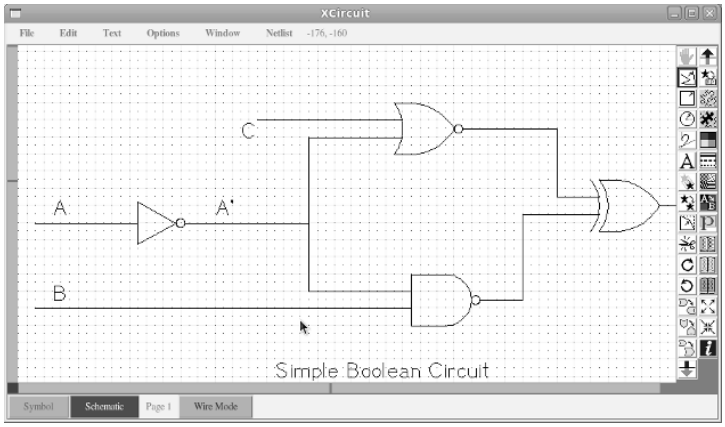
15.4.1 Xcircuit

The program xcircuit is a generic drawing program tailored especially for making publication-quality renderings of circuit diagrams (hence the name). The output is pure PostScript, and the graphical interface attempts to maintain as much consistency as possible between the X11 window rendering and the final printer output. An example of a simple circuit being drawn in Xcircuit is shown in Figure 15.2.

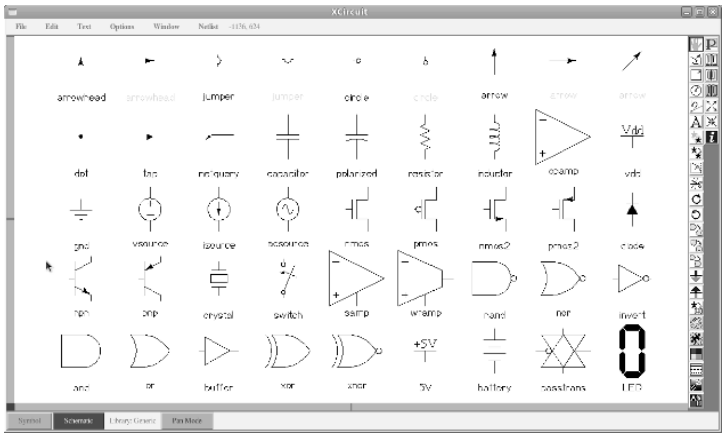
`Xcircuit` has five drawing elements:

1. polygon: (multiple lines which may or may not be closed and filled)
2. arc: (ellipse segment which may be closed and/or filled as above)
3. label: any text placed on the sheet,
4. curve: based on PostScript *curveto* command,
5. instance: object instance of another block.

Schematic representation (as produced from Xcircuit) of the Verilog design as shown in Listing 15.1 is shown in Figure 15.3(b).



(a) Simple boolean circuit



(b) Library elements

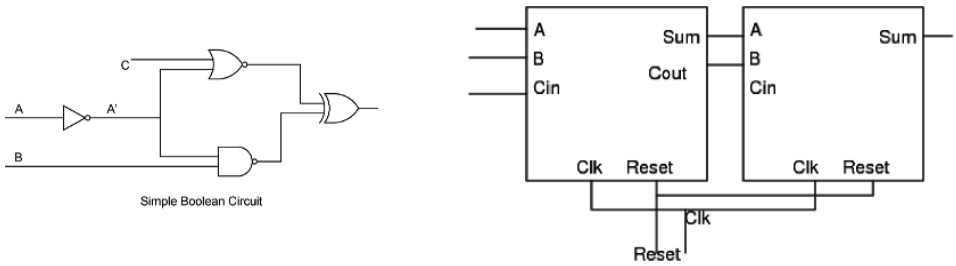
Fig. 15.2 Xcircuit : publication quality rendering of circuit schematics

15.4.2 GNU gschem

GNU `gschem` is part of gEDA suite of CAD tools. It is a full featured schematic editor. An example of `gschem` is shown in Figure 15.4. GNU `gschem` internally saves the schematic in its own format, but using the `gnetlist` command, SPICE netlist can be exported:

```
$gnetlist -g spice-sdb simple_circuit.sch
```

More details on SPICE processing is presented in Section 15.9.



(a) Simple boolean function (b) Verilog schematic

Fig. 15.3 Xcircuit used for drawing schematic

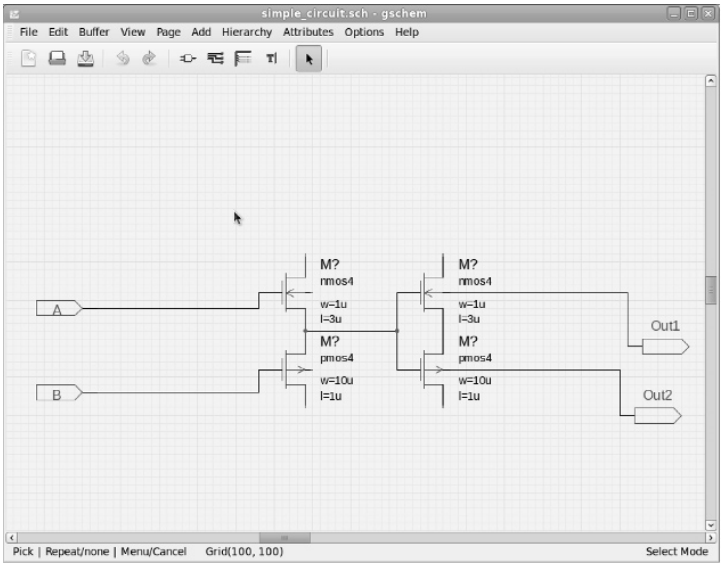


Fig. 15.4 gEDA schematic editor `gschem`

15.5 Verilog Processing

Verilog is a high-level hardward description language (HDL). Although it was originally designed for digital design simulation and verification, it is now also used

for design realization using *logic synthesis*. Alongwith, VHDL, Verilog is the dominant language for expressing digital logic. Structurally, Verilog is similar to C (by design), and VHDL has superficial resemblance to ADA. An example of a Verilog design (for a full adder) is shown in Listing 15.1.

```
/* \file adder.v
   \author Sandeep Koranne (C) 2010
   \description Example of Verilog file for adder
 */
module adder( A, B, Cin, Cout, Sum, Clk, Reset );
   input A,B,Cin,Clk,Reset;
   output Cout, Sum;
   reg    Cout, Sum;
   always @(posedge Clk or negedge Reset) begin
      if( Reset == 1'b0 ) begin
   {Cout,Sum} <= 0;
      end
      else begin
   {Cout,Sum} <= A + B + Cin;
      end
   end
endmodule // adder
```

Listing 15.1 Verilog design of full adder

Verilog is *timed*, whereas conventional C language has no notion of synchronization. Verilog (and indeed most other hardware description languages) describe *data flow* from *input* to *output*. The bit-width of expressions, and the operators which process these signals have to be carefully defined in Verilog as they have a strong impact on the *area* of the circuit produced. In C or other high level computer languages, the underlying execution machine is the CPU, which already has a predefined bus and register width (e.g., 32-bit or 64-bit). Thus, the cost of a 16-bit operation can be assumed to be the same as a 24-bit operation. In Verilog (and other HDLs) this is not the case, as the area of a 24-bit width data-path is significantly larger than a 16-bit datapath. Hardware designers use *design space exploration* to choose the smallest area producing circuit which meets the design goals.

Once a design has been created as a Verilog file, it can be simulated, synthesized and also tested for resilence against single *stuck-at* faults and delay faults. The specification of the design intent and its implementation should be carefully matched. In C or C++, the design intent is the design documentation, and its implementation if the executable. These are compared using tests. Since, HDLs undergo various transformations (logic synthesis, boolean optimizations, rewriting, retiming), the process of logic verification is important. Moreover, unlike C/C++ executables, where the fix to a detected problem may simply involve a patch, hardware circuits once implemented in chips, require expensive design *respins* costing millions of dollars. This provides additional impetus to the design verification step.

Open-source tools for HDL simulation and synthesis include the Verilog simulators Icarus and GPL `cver`, VHDL synthesis tools from the Alliance CAD suite. These are discussed below.

```
/* \file adder_tb.v
   \author Sandeep Koranne, (C) 2010
```

```
   \description Test bench for adder
 */

module adder_tb;
 reg A,B,Cin,Clk,Reset;
 wire temp;
 wire Cout,Sum;

 initial begin
  $dumpfile("adder_tb.vcd");
  $dumpvars(0,adder_tb);
  $monitor("Time = %d Clk = %b Sum = %d Cout = %d",
            $time, Clk, Sum, Cout );
  Clk <= 0;       A <= 0;
  B <= 0;       Cin <= 0;
  #20 Reset <= 0;
  #5  Reset <= 1;
  #10 A <= 1;
  #30 B <= 1;
 end

 always #10 Clk = ~Clk;
 always #100 $finish;
 adder U1(A,B,Cin,temp,,Clk,Reset);
 adder U2(temp, A, 0, Cout,Sum, Clk, Reset);
endmodule // adder_tb
```

15.5.1 Icarus Verilog Simulator

Icarus Verilog is a Verilog simulation and synthesis tool. It is a native code compiled Verilog simulator which generates machine code for Verilog models, the intermediate machine code is generated in a form called *vvp* assembly. This assembly code is executed on a virtual machine called *vvp*. The synthesis engine generates netlists in diverse formats including XNF (Xilinx Netlist Format). The compiler proper (or the compiler driver to be exact) is called *iverilog*. To prepare the above listed design for simulation we can run `iverilog adder.v`.

```
$ iverilog -h
Usage: iverilog [-ESvV] [-B base] [-c cmdfile|-f cmdfile]
              [-g1995|-g2001|-g2005] [-g<feature>]
              [-D macro[=defn]] [-I includedir]
              [-M depfile] [-m module][-N file]
              [-o filename] [-p flag=value][-s topmodule]
              [-t target] [-T min|typ|max] [-W class]
              [-y dir] [-Y suf] source_file(s)
```

```
$iverilog adder.v
```

A part of the generated VVP assembly is shown below for reference:

```
#! /usr/bin/vvp
:ivl_version "0.9.2 " "(v0_9_2)";
:vpi_time_precision + 0;
:vpi_module "system";
:vpi_module "v2005_math";
:vpi_module "va_math";
S_0x8ac00d8 .scope module, "adder" "adder" 2 5;
 .timescale 0 0;
v0x8ac0168_0 .net "A", 0 0, C4<z>; 0 drivers
v0x8ae0140_0 .net "B", 0 0, C4<z>; 0 drivers
v0x8ae01a0_0 .net "Cin", 0 0, C4<z>; 0 drivers
v0x8ae0200_0 .net "Clk", 0 0, C4<z>; 0 drivers
v0x8ae0268_0 .var "Cout", 0 0;
v0x8ae02c8_0 .net "Reset", 0 0, C4<z>; 0 drivers
v0x8ae0348_0 .var "Sum", 0 0;
E_0x8ae3da8/0 .event negedge, v0x8ae02c8_0;
E_0x8ae3da8/1 .event posedge, v0x8ae0200_0;
E_0x8ae3da8 .event/or E_0x8ae3da8/0, E_0x8ae3da8/1;
    .scope S_0x8ac00d8;
T_0 ;
    %wait E_0x8ae3da8;
    %load/v 8, v0x8ae02c8_0, 1;
    %cmpi/u 8, 0, 1;
    %jmp/0xz  T_0.0, 4;
    %ix/load 0, 1, 0;
    %assign/v0 v0x8ae0348_0, 0, 0;
    %ix/load 0, 1, 0;
    %assign/v0 v0x8ae0268_0, 0, 0;
    %jmp T_0.1;
T_0.0 ;
    %load/v 8, v0x8ac0168_0, 1;
    %mov 9, 0, 1;
    %load/v 10, v0x8ae0140_0, 1;
    %mov 11, 0, 1;
    %add 8, 10, 2;
    %load/v 10, v0x8ae01a0_0, 1;
    %mov 11, 0, 1;
    %add 8, 10, 2;
    %ix/load 0, 1, 0;
    %assign/v0 v0x8ae0348_0, 0, 8;
    %ix/load 0, 1, 0;
    %assign/v0 v0x8ae0268_0, 0, 9;
T_0.1 ;
    %jmp T_0;
    .thread T_0;
# The file index is used to find the file
# name in the following table.
:file_names 3;
    "N/A";
```

```
    "<interactive>";
    "adder.v";
```

The form of the genrated VVP assembly is instructive to understand the internal working of Icarus Verilog.

15.5.2 Pragmatic GPL `cver`

GPLCVER 2.11a is the name of a Verilog simulator written by Pragmatic C Software Corp and placed under GNU GPL. Simulating the Verilog design shown in Listing 15.1 gives us:

```
Compiling source file "adder.v"
Compiling source file "adder_tb.v"
Time =       0 Clk = 0 Sum = x Cout = x
Time =      10 Clk = 1 Sum = x Cout = x
Time =      20 Clk = 0 Sum = 0 Cout = 0
Time =      30 Clk = 1 Sum = 0 Cout = 0
Time =      40 Clk = 0 Sum = 0 Cout = 0
Time =      50 Clk = 1 Sum = 1 Cout = 0
Time =      60 Clk = 0 Sum = 1 Cout = 0
Time =      70 Clk = 1 Sum = 1 Cout = 0
Time =      80 Clk = 0 Sum = 1 Cout = 0
Time =      90 Clk = 1 Sum = 0 Cout = 1
Halted at location **adder_tb.v(26)
             time 100 from call to $finish.
  There were 0 error(s),
   2 warning(s), and 8 inform(s).
```

15.5.3 GTKWave: Waveform Viewer

The VCD (value change dump) file generated during the simulation can be viewed graphically using the GTKWave waveform viewer. Running GTKWave on the VCD file `gtkwave adder_tb.vcd` and adding the signals from instance U1 is shown in Figure 15.5.

15.6 VHDL Processing

Very high-speed integrated circuits hardware design language (VHDL) is a standardized language for describing the behavior and implementation of digital cir-

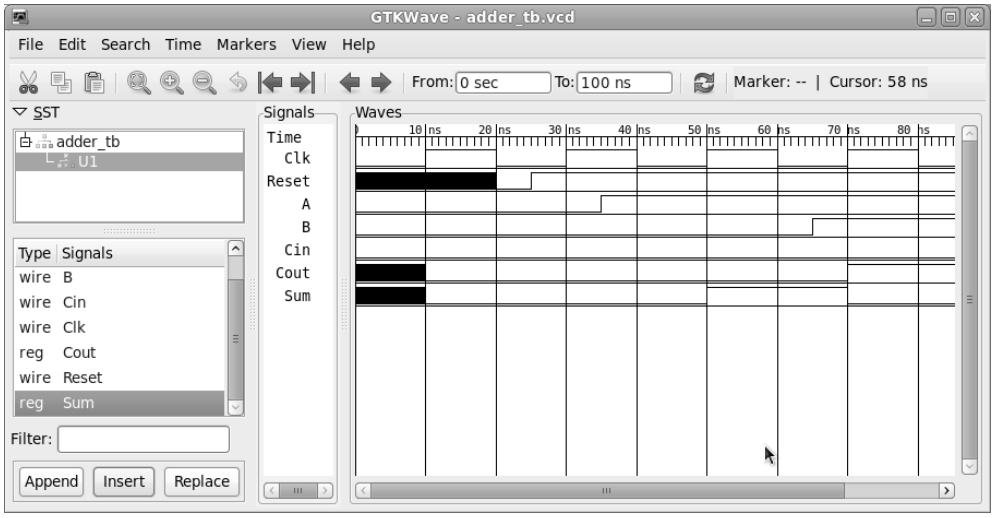


Fig. 15.5 GTKWave, waveform viewer from VCD files

cuits. It has syntactical resemblance to the ADA computer programming language. An example of an adder-accumulator is shown in Listing 15.2.

```
-- Adder-accumulator example in VHDL, from Alliance
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_arith.ALL;
use IEEE.STD_LOGIC_unsigned.ALL;


entity AddAccu is

    port ( CLK : in Std_Logic;
           CLR : in Std_Logic;
           LD  : in Std_Logic;
           A   : in Std_Logic_Vector(15 downto 0) ;
           RESULT : out Std_Logic_Vector(15 downto 0) );

end AddAccu;


architecture DataFlow OF AddAccu is
signal resultint : Std_Logic_Vector(15 downto 0) ;
begin
process (CLK)
begin
   if CLK'event and CLK='1' then
      if CLR = '1' then resultint  <= ( others => '0' );
      elsif LD = '1' then resultint <= resultint + A;
      end if;
 end if;
end process;
RESULT <= resultint;
end DataFlow;
```

Listing 15.2 VHDL example of adder-accumulator

We shall use the Alliance CAD system's VHDL simulator *asimut* to simulate this design.

15.7 Alliance CAD System

Alliance CAD is a complete VLSI CAD system. It includes tools for VHDL processing, schematic capture, logic synthesis, test pattern generation, layout editing, standard cell placement, routing, design rule checking (DRC), circuit extraction, layout versus schematic (LVS) comparison, and mask generation. We use the design shown in Listing 15.2 to examine the above mentioned CAD tools.

15.7.1 Alliance CAD VHDL processing

The input VHDL can include both structural (component interconnection) and behavioral statements. The Alliance CAD tool `vasy` (VHDL Analyzer for Synthesis) is used to process a given VHDL file. `vasy` accepts a subset of the complete VHDL language. VASY has a number of command line arguments which control its behavior. To write a '.vbe' behavioral file we add the '-a' argument, and the '-p' argument adds the power (VDD and VSS connectors). A complete list of the commandline arguments can be found by running `man vasy`.

```
vasy -a -B -o -p -I vhdl addaccu
```

The output of this tool is a '.vbe' file which expands any operators which are present in the VHDL file. This processed '.vbe' file can then be simulated using the VHDL simulator `asimut`.

15.7.2 Alliance CAD tool **`asimut`**

In Verilog, we had written an explicit test-bench module which contained the stimulus for the CLK, RESET, and the input signals. In Alliance CAD, the simulation *patterns* are listed in an external file. An example is shown below:

```
-- input / output list :
in        clk B;;
in        clr B;;
in        ld B;;
in        a (15 downto 0) X;;;
out       result (15 downto 0) X;;;

begin

<  0ns>  : 0 1 0 0000  ?****  ;
< +5ns>  : 0 1 0 0000  ?****  ;
< +5ns>  : 0 1 0 0000  ?****  ;
```

```
..
< +5ns>  : 1 0 1 0004  ?0004  ;
< +5ns>  : 0 0 1 0004  ?0004  ;
end;
```

The format of the pattern file is shown above. The list of input and output signals is followed by a sequence of time steps at which the input signals change values. In the above example, the `clk`, `clr`, and `ld` signal are 1 bit wide, whereas the input `A` signal is 15-bit. The advantage of using an external pattern file is that it can be generated from a program and can contain *expected* values for outputs. When simulated, any mismatch between the computed value and the value stored in the pattern file is treated as a simulation mismatch error and printed on the console.

The VHDL simulator in Alliance is `asimut`. It compiles and loads a complete hardware description written in VHDL . The hard- ware description may be structural (a hierarchy of instances) or behavioral. `asimut` also expects a pattern file as the input; the format of the pattern file was discussed above. The pattern file is compiled, loaded and linked with the hardware description, and the simulation is started.

```
$asimut -b addaccu addaccu res_vasy_1
..linking ...
executing ...
###----- processing pattern 0 : 0 ps -----###
###----- processing pattern 1 : 5000 ps -----###
###----- processing pattern 2 : 10000 ps -----###
..
```

The speed of the simulation can be increased by using BDD (binary decision diagrams), by specifying the `-bdd` commandline argument to `asimut`. Another advantage of using external pattern files is that post-synthesis gate level netlist can be simulated with the same pattern file. This provides a check for synthesis mismatch. The logic synthesis step of converting high-level VHDL circuit description to elementary gates is described next.

15.7.3 VHDL Logic Synthesis using Alliance CAD tool `Boom`

The Alliance CAD tool `Boom` is used for logic synthesis. The commandline for using `boom` is shown below:

```
$boom [options] [Algorithm] Input Output
```

The options available are:

```
 -V     Sets verbose mode on
 -T     Sets trace and verbose modes on
 -O     Reverses initial Bdd variables order
```

```
 -A     Keeps all auxiliary variables
 -P     Uses a parameter file (Input_file_name.boom)
 -L     Uses literal's number for surface estimation
 -l num Optimization level [0-3] (default 0, low level)
 -d num Delay optimization percent (default 0 %)
 -i num Iteration count
 -a num Amplitude
```

And the algorithm available to Boom are:

```
 -s     Simulated annealing (default)
 -j     Just do it algorithm
 -b     Burgun algorithm
 -g     Gradient algorithm
 -p     Procrastination algorithm
 -w     Window bdd reorder
 -t     Top bdd reorder
 -m     Simple bdd reorder
 -o     One pass (faster algorithm)
 -r     Random bdd reorder
 -n     No optimization algorithm
```

We use the `addaccu` example to show the logic synthesis procedure. Using `vasy` we had produced the '.vbe' file which is given as input to `boom`:

```
$boom -V -T -j addaccu.vbe
--> Parse BEH file addaccu.vbe

--> Check figure addaccu

--> Optimization parameters
    Algorithm : just do it
    Keep aux  : no
    Area      : 100 %
    Delay     :   0 %
    Level     :   0

--> Initial cost
    Surface  : 349750
    Depth    : 49
    Literals : 266

--> Translate Abl to Bdd
    Keep register signal resultint 14
    Keep signal rtlcarry_0 1

--> Optimization % 100
```

```
--> Final cost
    Surface  : 289500
    Depth    : 37
    Literals : 230

--> Post treat figure addaccu

--> Drive BEH file addaccu_o
```

The resulting '.vbe' file (addaccu_.vbe) consists of a gate level netlist. The *depth* of the circuit is 7, which means that there is atleast 1 path of 7 components from the input to output. Using different algorithms, the area cost of the circuit can be reduced, as well as a tradeoff between area and speed of the circuit performed. While the produced gate-level file is logically equivalent to the given input, it can be further optimized using Boolean transformations and mapped to a standard cell library. In Section 15.3 we had discussed the BLIF format and its specification of standard cell libraries. The impact of a good standard cell library function in reducing area and delay was shown in Figure 15.1, where the same logical function was implemented in smaller area by using a more complex standard cell. In Alliance, standard cell libraries are defined using Catalog files, and the library mapper tool is called `boog`. `boog` takes as input the gate-level '.vbe' file, and the target library. It produces a gate-level netlist.

```
boog <input_file> [-o <output_file>] [-l <lax_file>]
                  [-x <xsch_mode>] [-m <optim_mode>]
MBK_TARGET_LIB : /usr/lib/alliance/cells/sxlib
```

We use `boog` to *map* the addaccu_o.vbe gate-level file to a netlist of cells from the Alliance `sxlib` standard cell library:

```
$boog addaccu_o -o addaccu -x 1 -m 2
```

We specify the optimization mode (using `-m 2`) as well as generate a highlighted schematic file (the critical path is marked) using the `-x 1` commandline arguments. The input and output file names are also given to the tool.

```
Reading default parameter...
50% area - 50% delay optimization
Reading file 'addaccu_o.vbe'...
Controlling file 'addaccu_o.vbe'...
Reading lib '/usr/lib/alliance/cells/sxlib'...
Preparing file 'addaccu_o.vbe'...
Capacitances on file 'addaccu_o.vbe'...
Unflattening file 'addaccu_o.vbe'...
Mapping file 'addaccu_o.vbe'...
Saving file 'addaccu.vst'...
Adding signal 'not_a 3'
```

```
Adding signal 'not_a 7'
Quick estimated area
   (with over-cell routing)...265250 lambda
Details...
inv_x2: 36
xr2_x1: 31
buf_x2: 16
sff1_x4: 16
oa2a22_x2: 16
...
no2_x1: 1
Total: 144
Saving delay gradient in xsch color file
'addaccu.xsc'...
End of boog...
```

The standard cell mapping produces a '.vst' file (addaccu.vst) as well as a schematic tile (addaccu.xsc).

15.7.4 Alliance CAD tool `xsch` schematic viewer

We have seen the XCircuit tool for schematic depiction in Section 15.4. The Alliance CAD tool for schematic viewing is called `xsch`. Using the generated 'addaccu.xsc' file from `boog` we can run the schematic viewer as:

```
$xsch -l addaccu
```

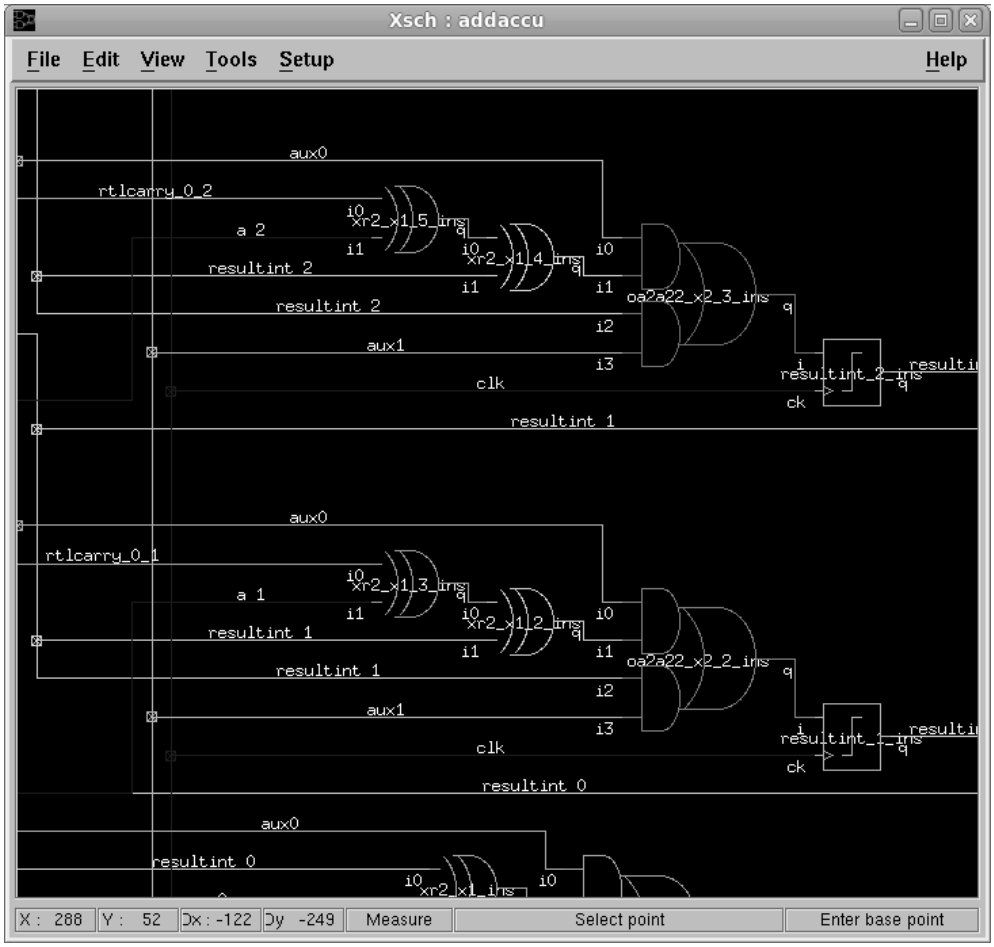
The produced schematic plots are shown in Figure 15.6. The critical path is marked. `xsch` can also generate Xfig '.fig' format files. Xfig is a vector graphics editor (see Section 19.7).

In addition to the schematic file, the main output of `boog` is the gate-level netlist which represents our adder-accumulator circuit in terms of standard cell library elements. This gate-level netlist is optimized and converted to physical geometry in the next sections.

15.7.5 Gate level processing in Alliance CAD

The gate-level netlist format used in Alliance is structural VHDL. This is a subset of VHDL used for component netlisting. An example is shown below:

```
entity addaccu is
   port (
      clk    : in      bit;
```



(a) Instances

Fig. 15.6 Schematic viewer tool in Alliance CAD `xsch`

```
      clr    : in      bit;
      ld     : in      bit;
      a      : in      bit_vector(15 downto 0);
      result : out     bit_vector(15 downto 0);
      vdd    : in      bit;
      vss    : in      bit
 );
end addaccu;

architecture structural of addaccu is
Component no2_x1
   port (
      i0  : in      bit;
      i1  : in      bit;
```

```
      nq  : out     bit;
      vdd : in      bit;
      vss : in      bit
 );
end component;
....
not_rtlcarry_0_12_ins : inv_x2
   port map (
      i   => rtlcarry_0_12,
      nq  => not_rtlcarry_0_12,
      vdd => vdd,
      vss => vss
   );

not_resultint_11_ins : inv_x2
   port map (
      i   => resultint(11),
      nq  => not_resultint(11),
      vdd => vdd,
      vss => vss
   );
end structural;
```

The gate-level '.vst' file declares an *entity* which is the top-level design `addaccu`. Subsequently, every standard cell which is instanced in this design is *declared*. Then a *structural* architecture of the design is defined as the interconnection of instances. The interconnections are made using *signals* which are equivalent to wires connection outputs of components to inputs of other instances. This gate-level netlist is actually a collection of directed acyclic graphs from the *primary inputs* of the design, to *primary outputs* of the design. The *critical path* in this graph is the path from an input to output which has the maximum cumulative delay on it. To increase the speed of the circuit, this path delay should be minimized. In VLSI designs, increasing the area of the driver cells can reduce the path delay (but at a cost of increasing area). This analysis, of which cells to *resize* is complex, and dependent on the graph theoretic analysis of the design, as well as transistor level analysis of the standard cells. The Alliance CAD tool `loon` (local optimization on nets) performs this optimization. `loon` takes as input the gate-level .vst file and produces an equivalent (but optimized) gate-level netlist.

```
$loon addaccu addaccu_o
Reading default parameter...
50% area - 50% delay optimization
Reading file 'addaccu.vst'...
Reading lib '/usr/lib/alliance/cells/sxlib'...
Capacitances on file 'addaccu.vst'...
Delays on file 'addaccu.vst'...13297 ps
Area on file 'addaccu.vst'...
  265250 lamda (with over-cell routing)
```

```
Details...
inv_x2: 36 (10%)
xr2_x1: 31 (26%)
buf_x2: 16 (6%)
sff1_x4: 16 (27%)
oa2a22_x2: 16 (13%)
a2_x2: 13 (6%)
nao2o22_x1: 12 (7%)
oa2ao222_x2: 2 (1%)
an12_x1: 1 (0%)
no2_x1: 1 (0%)
Total: 144
Worst RC on file 'addaccu.vst'...438 ps
Inserting buffers on critical path
...15 buffers inserted -> 12698 ps
Improving RC on critical path for file
  'addaccu_o.vst'...12558 ps
Improving all RC for file 'addaccu_o.vst'...
Worst RC on file 'addaccu_o.vst'...438 ps
Area on file 'addaccu_o.vst'...
  281250 lamda (with over-cell routing)
Details...
inv_x2: 33 (8%)
xr2_x1: 31 (24%)
buf_x2: 30 (10%)
sff1_x4: 16 (25%)
oa2a22_x2: 16 (12%)
a2_x2: 13 (5%)
nao2o22_x1: 12 (7%)
inv_x4: 3 (1%)
oa2ao222_x2: 2 (1%)
buf_x4: 1 (0%)
an12_x1: 1 (0%)
no2_x1: 1 (0%)
Total: 159
Critical path (no warranty)...12558 ps
  from 'resultint 0' to 'resultint_15_ins'
Saving file 'addaccu_o.vst'...
```

The `loon` tool has resized some of the cells on the critical path to use larger drive strength variants. It can also add inverter-pairs as buffers on some paths. This optimized gate-level netlist is the handoff from the front-end design team to the back-end team which converts this textual information to physical mask geometry which is then sent off to the VLSI fab house for manufacturing as computer chips.

15.7.6 Physical design with Alliance CAD

The first step in converting a gate-level standard cell netlist to manufacturable geometry is *standard cell placement*. To aid in automatic placement, all standard cells have the same height (thats why they are called standard). A standard cell placer arranges these cells in *rows* while minimizing the wire length. Placement is a hard combinatorial problem, and therefore advanced techniques have been developed to approximate a good solution. The Alliance CAD tool's standard cell placer is called `ocp` and implements a simulated-annealing based placer. In addition to placing the cells, `ocp` can also perform IO ring placement, from a given IO constraint file:

```
TOP ( # IOs are ordered from left to right
   (IOPIN clk.0 );
   (IOPIN clr.0 );
   (IOPIN ld.0 );
   (IOPIN a(15).0 );
)
BOTTOM ( # IOs are ordered from left to right
   (IOPIN result(15).0 );
)
IGNORE ( # IOs are ignored(not placed) by IO Placer
)
```

The number of rows can be input to the placer program. We run `ocp` on our design:

```
$ocp -v -gnuplot -ioc addaccu_o addaccu addaccu_p

 o Number total of instances is ....  144
 o Number of instances to place is ....  144
 o Number of instances already placed is ....  0
 o Number of nets is .... 170
 o Sum of instances to place widths is ... 1061
 o Computing Initial Placement ...
 o User Margin : 20%
 o Number of Rows : 11
 o Real Margin : 16.8495%
 o Width of the abutment box : 116
 o Height of the abutment box : 110
 o conspace : 6.10526 1st connector : 3.05263
 o adding connector  : clk x : 3 y : 110
 o adding connector  : clr x : 9 y : 110
 o adding connector  : ld x : 15 y : 110
 o adding connector  : a 15 x : 21 y : 110
 o Initial Placement Computing ... done.
 o Beginning global placement ....
 Loop = 1, Temperature = 0.245678, Cost = 0.959008
```

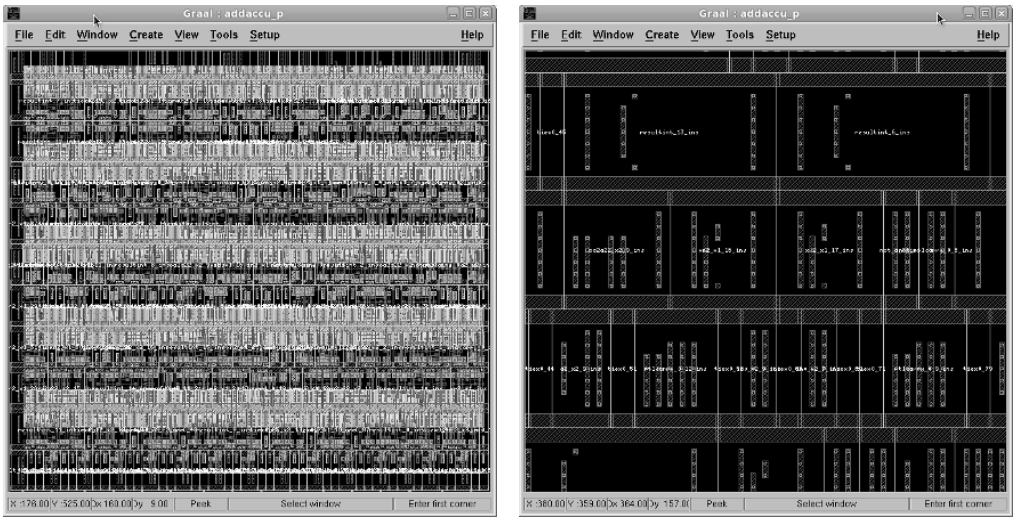
```
  RowCost = 129.455, BinCost = 260.597, NetCost = 15557.5
  Success Ratio = 99.5688%, Dist = 1, Delta = 0.5
 o Total impossible movements = 650
 o 3.84615 % suroccupied target
 o Final Optimization succeeded ...
 o Final Net Cost ..... 5094.5
 o Final Net Cost Optimization ..... 23.7407%
 o Total Net Optimization .... 68.5961%

Ocp : placement finished
```

The produced placement is a coordinate assignment of (X,Y) for every instance of the netlist:

```
V ALLIANCE : 6
H addaccu_p,P,13/ 7/2010,100
A 0,0,58000,55000
I 46000,0,tie_x0,tiex0_93,SYM_Y
I 44000,45000,tie_x0,tiex0_92,NOSYM
I 43000,0,tie_x0,tiex0_91,SYM_Y
I 42000,50000,tie_x0,tiex0_90,SYM_Y
...
```

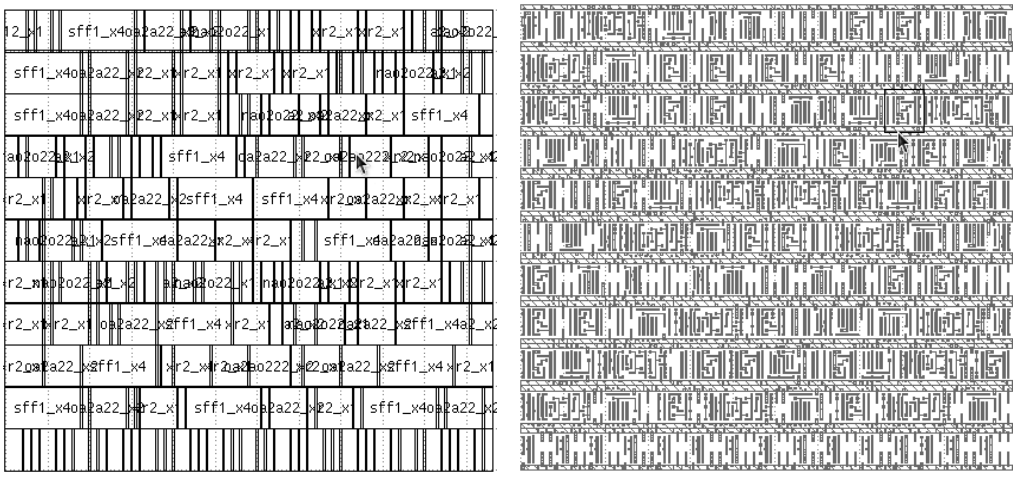
The placement can be viewed using the graphical symbolic viewer tool in Alliance CAD `graal` as shown in Figure 15.7.



(a) Cell placement in `graal` (b) Cell placement in zoomed in

Fig. 15.7 Alliance CAD tool `graal` showing OCP placement

A more detailed view of the placement coordinates is shown in Figure 15.8(a), and the metal interconnect layer which lines up in the standard cell rows is shown in Figure 15.8(b).



(a) Cell coordinates (b) Metal interconnect for rows

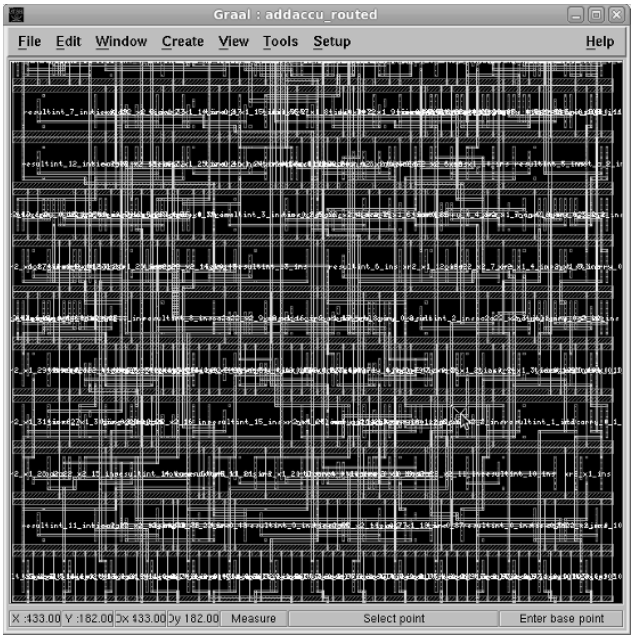
Fig. 15.8 Standard cell placement showing cell blocks and metal interconnect

15.7.7 Alliance CAD tool for standard-cell routing: `nero`

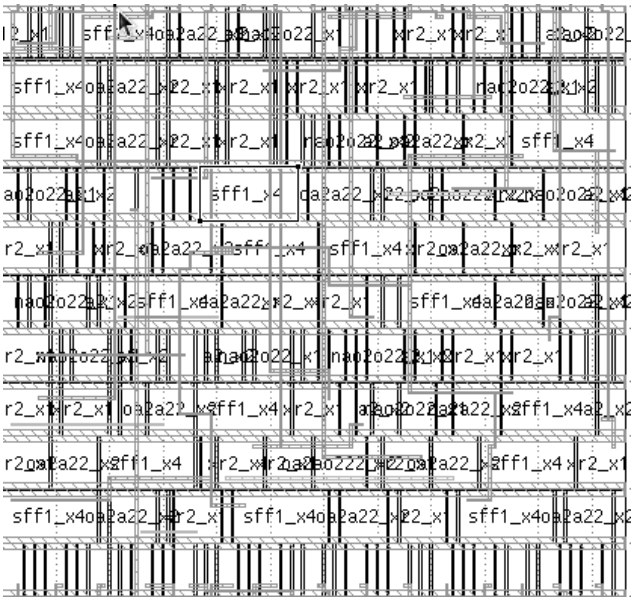
The standard-cell router in Alliance CAD tools is called `nero`, for *Negotiating Router*. It takes as input the number of metal tracks to use for routing, the design file containing cell coordinates (placement output) and the gate level netlist. It produces a symbolic placement file containing valid track assignments for metal wires which connect the various components of the netlist together. VLSI routing is another hard combinatoric problem, and the router implemented in Alliance CAD uses *negotiation*, and priority ordering to converge to a valid routing solution. The output of the router is shown in Figure 15.9(a) and Figure 15.9(b).

Till this point, the physical geometries are maintained in symbolic, or *lambda* form. The advantage of symbolic geometry representation is the ease of technology migration, scaling, design rule verification and circuit extraction. The tool for circuit extraction is `cougar` which takes as input the post routed layout, and extracts the netlist of interconnections (including parasitic resistance and capacitance). The extracted netlist can be in hierarchical SPICE format:

```
* Spice description of addaccu_e
* Spice driver version 11952808
* Date ( dd/mm/yyyy hh:mm:ss ): 13/07/2010 at 16:12:02
```



(a) Routing output in `graal`



(b) Metal track for routing

Fig. 15.9 Output from `nero` negotiating router

```
* INTERF a[0] a[1] a[2] .. a[11] a[15]
* INTERF clk clr ld
* INTERF result[0] ... result[15]
* INTERF vdd vss
```

```
.INCLUDE rowend_x0.spi
.INCLUDE tie_x0.spi
...
C102 rtlcarry_0_3 vss 8.90499e-15
C101 a[3] vss 2.31066e-14
C100 xr2_x1_7_sig vss 6.83829e-15
C99 mbk_buf_rtlcarry_0_3 vss 6.7001e-15
C1 result[15] vss 3.77352e-15
.ends addaccu_e
```

The extracted SPICE netlist can be analyzed using `ngspice` SPICE simulator. See Section 15.9 for more details on using SPICE simulators.

Using the Alliance CAD tool `s2r` (symbolic to real), symbolic geometries are converted to actual mask data. The tool can moreover perform post translation treatment of the layout.

15.7.8 QUCS : Universal Circuit Simulator

QUCS is an integrated electronic circuit simulator. An example of a simple digital circuit designed in QUCS is shown in Figure 15.10.

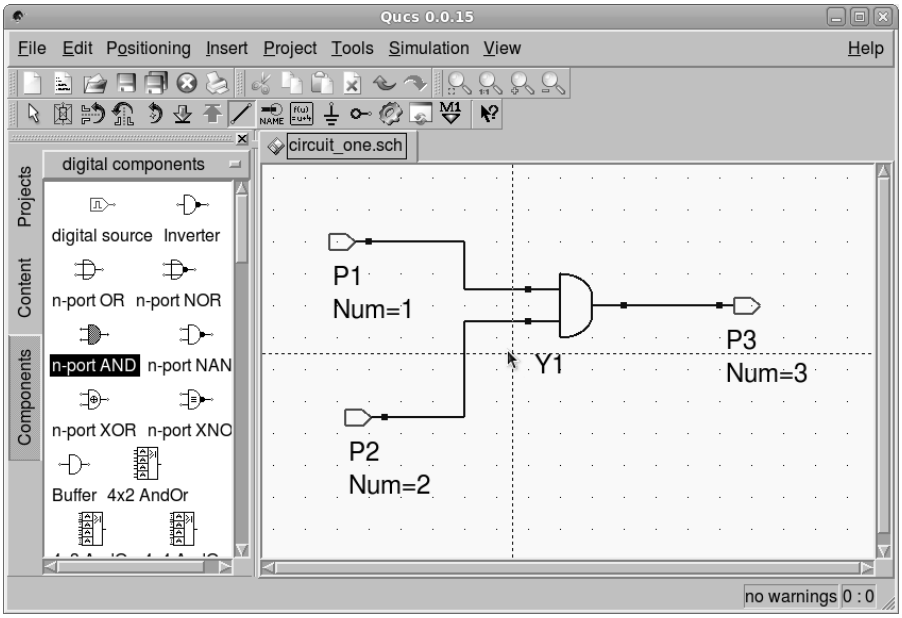


Fig. 15.10 Example of digital circuit designed in QUCS

The circuit can be simulated within QUCS, and even DC, AC and S-parameter analysis can be performed. The schematic design format of QUCS is XML:

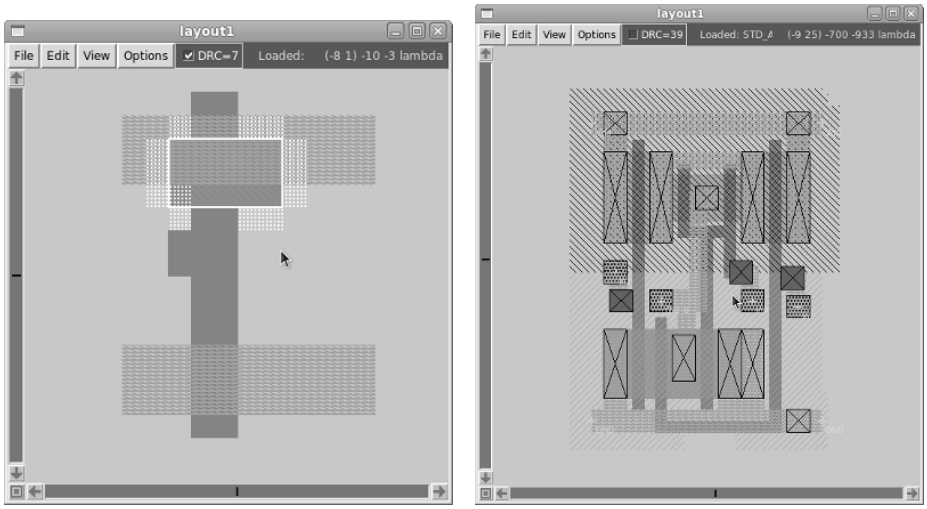
```
<Qucs Schematic 0.0.15>
```

```
<Properties>
  <View=13,12,396,265,1.5,0,0>
  <Grid=10,10,1>
..
</Properties>
<Components>
  <Port P3 1 300 100 4 12 1 2 "3" 1 "out" 0>
  <Port P1 1 80 60 -23 12 0 0 "1" 1 "in" 0>
  <Port P2 1 90 170 -23 12 0 0 "2" 1 "in" 0>
  <AND Y1 1 210 100 -26 27 0 0 "2" 0 "1 V" 0 "0" 0 "10" 0 "old" 0>
</Components>
```

15.8 Magic VLSI Editor

Magic is a full custom VLSI layout editor.

It can represent Manhattan mask geometry for a wide variety of process nodes, and uses a clever *corner stitched* data representation alongwith layer *tiling* which allows Magic to perform design-rule checking (DRC) and layout extraction very efficiently. Another advantage of Magic is that it represents the design geometry as simple TEXT files. Consider a small layout example shown in Figure 15.11.



(a) Partial layout in Magic (b) Example of Standard cell

Fig. 15.11 Magic VLSI layout editor

The same layout in Magic's layout file is shown below:

```
magic
tech scmos
timestamp 1279089463
<< error_s >>
```

```
rect 2 7 3 8
rect 5 7 7 8
rect 1 5 2 7
rect 1 4 3 5
rect 7 4 8 7
rect 2 3 3 4
rect 5 3 7 4
<< polysilicon >>
rect 3 7 5 9
rect 3 3 5 4
rect 2 1 5 3
rect 3 -6 5 1
<< metal1 >>
rect 0 5 11 8
rect 0 -5 11 -2
<< ntransistor >>
rect 2 4 7 7
<< end >>
```

15.9 NGSpice SPICE Engine

SPICE is an acronym for simulation program with integraged circuit emphasis. `ngspice` is a general purpose circuit simulator for linear and non-linear circuit analysis. It is an update of the Berkeley Spice 3f5 program. `ngspice` can perform: (i) analog simulation, (ii) mixed-mode simulation as well as (iii) digital circuit simulation. The digital simulation model takes advantage of *event propagation* to optimize the computation. `ngspice` supports the following types of analysis:

1. DC analysis: operating point and DC sweep, with inductors shorted and capacitors opened. A DC analysis is automatically performed before transient analysis,
2. AC small signal analysis:limited to analog nodes, and represents the steady-state behavior at a particular set of stimulus frequencies,
3. Transient analysis: extension of DC analysis to the time domain,
4. Pole-zero analysis: computes poles and zeros in the small-signal AC transfer function,
5. Small-signal distortion analysis: computes steady-state harmonics,
6. Sensitivity analysis: calculates either the DC operating-point sensitivity, or AC small-signal sensitivity,
7. Noise analysis: analyzes device generated noise.

Circuits in SPICE are described using topological conventions: (i) circuit cannot contain a loop of voltage sources, (ii) each node in the circuit must have a dc path to ground, and (iii) every node must have at least two connections. A simple SPICE circuit is shown in Figure 15.12.

The circuit shown in Figure 15.12 can be defined in SPICE as:

```
* Example of a simple circuit in Spice
VS 1 0 DC 10V
```

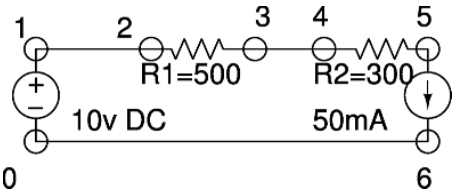


Fig. 15.12 Example circuit for SPICE analysis

```
IS 5 0 DC 50MA
R1 1 3 500
R2 3 5 300
.END
```

The circuit has a voltage source between nodes 1 and 0 of 10 volts, and a current source of 50 mA between nodes 5 and 0. The two resistors are placed in series. The SPICe file contains a title, and the last line contains `.end` to depict end of circuit. Hierarchical SPICE netlists can be generated:

```
.subckt VMODULE 1 2 3 ; defines a subcircuit
r1 1 2 500
r2 2 3 100
.ends

U1 5 6 9 VMODULE ; refers to an instance of VMODULE
```

15.9.1 Elementary devices in SPICE

SPICE already knows about elementary devices such as resistors, capacitors, voltage sources, current sources and MOSFETS. Resistors are introduced using the `R<n>` notation, capacitors with the `C<n>` notation, inductors with `L<n>`, coupled inductors with `K<n>`, switches with `S<n>`, voltage sources with `VS<n>`, current sources with `IS<n>` and MOSFETS with `M<n>`. Diodes are represented with `D<n>` and BJT models with `Q<n>`.

A MOSFET model is defined as:

```
M<n> nd ng ns nb mname <m=val> <l=val> <w=val>
```

where *nd* denotes the connection to *drain*, *ng* connection to *gate*, *ns* connection to *source*. The model name is given in `mname`, and device parameters *m* denotes multiplicity, while *l* and *w* denote the *length* and *width* of the MOSFET channel, respectively. `ngspice` supports advanced device models in MOSFETS, including the BSIM1, and BSIM3 models.

15.9.2 Performing TRANSIENT analysis

An example of performing transient analysis on the above circuit is shown below.

```
ngspice 1 -> source ex1.cir
Circuit: * example of a simple circuit in spice
ngspice 2 -> tran 1ns 10ns
Doing analysis at TEMP = 27.000000 and TNOM = 27.000000
Initial Transient Solution
--------------------------
Node                                   Voltage
----                                   -------
1                                           10
5                                          -30
3                                          -15
vs#branch                                -0.05
%100.00
No. of Data Rows : 60
```

ngspice also has the ability to plot signal waveforms directly from the interactive command terminal.

15.9.3 Conclusion

In this section we have discussed the many open-source tools we have for doing VLSI CAD. Verilog, VHDL simulation, logic synthesis, placement, routing, schematic generation, physical layout editing and SPICE simulation were discussed in this section. In addition to VLSI CAD, some of the software libraries developed for VLSI have also found use in othe domains. For example, Tcl/Tk were initially developed for glue logic between CAD tools, binary-decision diagram libraries for logic synthesis and verification are also used in combinatoric search, and satisfiability testing is also used as a search space exploration tool in integer linear programming.